Case - Normatic (Documentation)

I will be using Codesys 4 for the coding environment as I'm more familiar with it, and the alternative is not available on my home system as I'm running Linux CachyOS.

Task 0

Following RFC 2324 HTCPCP (Hyper Text Coffee Pot Control Protocol) 1.0 to fulfil the scope of the task to a satisfying degree.
Tastes good, will repeat as needed.

Task 1

The goal of the task is to scale an input in the range of 0 to 3 meters to an output of 4 to 20 mA. I will be using datatype REAL for the inputs and outputs to get a higher resolution. I am using a method calling Linear Mapping to scale the value, and I'm making it so that it can be used in other projects as well.

Linear Mapping: Take the ratio of the input range, multiply it with the output range and add the minimum for the output to get the scaled value.

$$ScaledValue = \frac{Input - InputMin}{InputMax - InputMin} \times (OutputMax - OutputMin) + OutputMin$$

I made a function that does the calculation

Function

```
FUNCTION SCALE : REAL

VAR_INPUT



rInput : REAL; //Input

rInputMin, rInputMax : REAL; //Min and max value of inputs

rOutputMin, rOutputMax : REAL; //Min and max value of outputs



END_VAR

VAR



_rRangeInput : REAL; //Range of input

_rRangeOutput : REAL; //Range of output

_rInputRatio : REAL; //Ratio of input to input range

END_VAR
```

```
//This function block scales input to match a range defined in the initialization

//Getting the ranges of the input and output

_rRangeInput := rInputMax - rInputMin;

_rRangeOutput := rOutputMax - rOutputMin;



//Getting the ratio of the input to the input range

_rInputRatio := (rInput - rInputMin) / _rRangeInput;



//Calculating the scaled value

SCALE := (_rInputRatio * _rRangeOutput) + rOutputMin;



END_FUNCTION
```

Task 2

The task asks for a function block that starts and stops the UV system based on the level of the tank.

This is done by taking the calculated water level from task 1 as an input, and comparing it to a high and low value. I made a FB_init method to create these high and low values when the function block is declared.

FB_UVController:

```
FUNCTION_BLOCK FB_UVController

VAR_INPUT

rWaterLevel : REAL; //The level of water in the tank

END_VAR

VAR_OUTPUT

xUVFilterStatus : BOOL; //Output for on/off for the UV Filter

END_VAR

VAR

_rWaterLowAlarm := FALSE REAL; //The lowest acceptable water level
```

```
_rWaterHighAlarm := FALSE REAL; //The highest acceptable water level

END_VAR



IF rWaterLevel < _rWaterLowAlarm THEN //When water level is too low, UV System needs
to run

xUVFilterStatus := TRUE;

ELSE IF rWaterLevel > _rWaterHighAlarm THEN //When water level is too high, UV
System needs to shut down

xUVFilterStatus := FALSE;

END_IF



END_FUNCTION_BLOCK
```

FB_init

```
//FB_Init is always available implicitly and it is used primarily for
initialization.

//The return value is not evaluated. For a specific influence, you can also declare
the

//methods explicitly and provide additional code there with the standard
initialization

//code. You can evaluate the return value.

METHOD FB_Init : BOOL

VAR_INPUT

bInitRetains : BOOL; // TRUE: the retain variables are initialized (reset warm /
reset cold)

bInCopyCode : BOOL; // TRUE: the instance will be copied to the copy code afterward
(online change)

rWaterLowAlarm : REAL; //Lowest acceptable water level

rWaterHighAlarm : REAL; //Highest acceptable water level

END_VAR
```

```
//Setting the alarm levels

_rWaterLowAlarm := rWaterLowAlarm;

_rWaterHighAlarm := rWaterHighAlarm;



END_METHOD
```

Task 3

This task ask to create logic controlling the output pumps. They only need one to run at a time, and it should alternate. There is also fault handling, if the pump is experiencing fault it should not run, and also it should not try to start the faulty pump.
I will make a function block, so that it will be easier to reuse the code at a later date, if needed.

The logic is using nested IF sentences to make sure the priority is right, and there is no fault in the engine trying to be started.

```
FUNCTION_BLOCK FB_PumpController

VAR_INPUT

xEngine1Fault : BOOL; //Fault of engine 1

xEngine2Fault : BOOL; //Fault of engine 2

xRunPumps : BOOL; //True: Run pumps. False: Stop pumps

iPriority : INT := 0; //Which engine should run



END_VAR

VAR_OUTPUT

xEngine1Run : BOOL; //True: Run engine 1

xEngine2Run : BOOL; //True: Run engine 2

END_VAR

VAR

END_VAR




IF xRunPumps THEN //Check if the pumps have run signal

IF iPriority = 0 THEN // pump 1 priority
```

```
IF NOT xEngine1Fault THEN //No fault detected on engine 1

xEngine1Run := TRUE; //Start engine 1

xEngine2Run := FALSE; //Stop engine 2

ELSIF NOT xEngine2Fault THEN //If fault detected on engine 1, try engine 2

xEngine2Run := TRUE; //Start engine 2

xEngine1Run := FALSE; //Stop engine 1

END_IF

ELSE // pump 2 priority

IF NOT xEngine2Fault THEN // No fault detected on engine 2

xEngine2Run := TRUE; //Start engine 2

xEngine1Run := FALSE; //Stop engine 1

ELSIF NOT xEngine1Fault THEN //If fault detected on engine 2, try engine 1

xEngine1Run := TRUE; //Start engine 1

xEngine2Run := FALSE; //Stop engine 2

END_IF

END_IF

ELSE

xEngine1Run := FALSE;

xEngine2Run := FALSE;

END_IF




END_FUNCTION_BLOCK
```

Task 4

The task asks for a function block to start and stop the UV filtration system based on the level of the tank, LT01. Task 2 already created the function block to detect whether or not the UV filter needs to be turned on or off. This function block therefore does not need a way to check the level of the water.

Turning on needs to happen sequentially when reading low levels. The UV needs to start and spend some time heating up before the valve can be opened.
Send signal to start UV -> Wait for UV to be ready -> Open valve

Turning off the UV system also needs to happen sequentially. The valve needs to be completely shut off before the UV systems goes down.
Send signal to shut valve -> Wait for valve to be shut down -> Shut down UV system

To make this happen i will be using a state machine. The different states are:

State	State ID	Action
Standby	00	Detect low/high levels
Detect low level	10	Turn on UV filter
UV Filter heating up	11	UV Filter is ready
UV Filter is ready	12	Turn on valve
Standby	00	Detect low/high levels
Detect high level	20	Close valve
Valve is closing	21	Valve is closed
Valve is closed	22	Turn off UV filter

Pseudocode

```
Switch(State)
     Case (00) //Standby mode, check for water level
          IF (WaterLevel == LOW)
               State := 10;
          Else if (WaterLevel == HIGH)
               State := 20;
     Case (10) //Low water detected, turn on UV filter
          TurnOnUVFilter();
          State := 11;
     Case (11) //Wait for UV filter to turn on
          If(UVFilter == TRUE) //UV filter returns true when ready
               State := 12;
     Case (12) //UV filter is ready to recieve water
          OpenValve();
          State := 00; //Back to standby mode
     Case (20) //High water detected, close valve
          CloseValve();
          State := 21;
     Case (21) // Waiting for valve to close
          If(Valve == TRUE) //Valve returns true when closed
               State := 22;
     Case (22) // Valve is closed, ready to shut down UV filter
          TurnOffUVFilter();
          State := 00; //Back to standby mode
End SwitchCase
```

FB_ValveController

```
FUNCTION_BLOCK FB_ValveController

VAR_INPUT

xValveStatus : BOOL; //State of valve. TRUE = Confirmed shut

xUVStatus : BOOL; //State of UV heat. TRUE = Ready

xFilterStatus : BOOL; //State of filter. Comes from UV controller



END_VAR

VAR_OUTPUT

xUvFilter : BOOL; //True = Turn on. False = Turn off

xValve : BOOL; //True = Open. False = Close



END_VAR

VAR

_rWaterLowAlarm : REAL; //Lowest acceptable water level

_rWaterHighAlarm : REAL; //Highest acceptable water level

_iState : INT := 0; //State of the state machine.

_xLastFilterStatus : BOOL := FALSE; //The last filter status, used for edge
detection

END_VAR



CASE _iState OF

0: //Standby mode, waits for change in filter status

IF _xLastFilterStatus <> xFilterStatus THEN // Detect change in status of UV filter

IF xFilterStatus THEN

//Filter needs to turn on

_iState := 10;

ELSE

//Filter needs to turn off
```

```
_iState := 20;

END_IF

END_IF

10: //Starting the filter system

xUvFilter := TRUE; //Starting UV filter heater

_iState := 11;

11: //Waiting for UV filter to heat up

IF xUVStatus THEN

//UV Filter has reached sufficient temperature

_iState := 12;

END_IF

12:

xValve := TRUE; //Open the valve

_iState := 0; //Returns to standby mode

20: //Closing the filter system and shutting it off._3S_LICENSE

xValve := FALSE; //Closes the valve

_iState := 21;

21: //Waiting for valve to close

if xValveStatus THEN

//Valve has closed

_iState := 22;

END_IF

22:

xUvFilter := FALSE; //Shutting off the UV Filter._3S_LICENSE

_iState := 0; //Returns to standby mode



END_CASE
```

```
//Updating the last filter status

_xLastFilterStatus := xFilterStatus;



END_FUNCTION_BLOCK
```

PLC_PRG

Here I will paste the main code, with all the testing variables I have used.

```
PROGRAM PLC_PRG

VAR

//IO List



// Clean water tank transmitter 01

LT01 : REAL;

//(Closed) Inlet valve for raw water, 1(TRUE) when the valve is confirmed closed

//(Activate) Inlet valve for raw water, activates the Valve, the valve is fully open
when this is 1 (TRUE)

V01 : BOOL;

//(Running) UV-filter for raw water, 1(TRUE) when the filter is operational and
ready for raw water

//(Activate) UV-filter for raw water, 1(TRUE), starts the UV

UV01 : BOOL;



//(Fault) P01 has a severe fault and cannot be started

//(Activate) P01 activation, when high the P01 is running unless Fault

P01 : BOOL;



//(Fault) P02 has a severe fault and cannot be started

//(Activate) P02 activation, when high the P02 is running unless Fault
```

```
P02 : BOOL;


// Initializing the Function Block

UVController : FB_UVController(rWaterLowAlarm := 1, rWaterHighAlarm := 2);

ValveController : FB_ValveController;

PumpController : FB_PumpController;


//Variables

LT01Calc : REAL; //Calculated level of the tank

xFilterOnOff : BOOL; //Whether or not the filter should be running or not

iPumpPriority : INT := 1; //Pump priority, used to cycle between pumps. 0 = pump 1,
1 = pump 2



//Test Variables

rLT01_test : REAL := 20;

rInputMax : REAL := 20;

rInputMin : REAL := 4;

rOutputMax : REAL := 3;

rOutputMin : REAL := 0;



UV01_running : BOOL := TRUE;

UV01_activate : BOOL;

V01_closed : BOOL := TRUE;

V01_activate : BOOL;


P01_fault : BOOL;

P01_activate : BOOL;
```

```
P02_fault : BOOL := TRUE;

P02_activate : BOOL;



xRunThePumps : BOOL := TRUE;



END_VAR



//Calculates the level in the tank



LT01Calc := SCALE(rLT01_test, rInputMin, rInputMax, rOutputMin, rOutputMax);



//Checking if the level is too high or low

UVController(rWaterLevel := LT01Calc);

xFilterOnOff := UVController.xUVFilterStatus; //Updating status of whether the
filter should be on or off



//Operating the pumps

//Updating the pump values

PumpController(xEngine1Fault := P01_fault, xEngine2Fault := P02_fault, xRunPumps :=
TRUE, iPriority := iPumpPriority);



//Updating the IO for the pumps

P01_activate := PumpController.xEngine1Run;

P02_activate := PumpController.xEngine2Run;



//Updating the ValveController

ValveController(xValveStatus := V01_closed, xUVStatus := UV01_running, xFilterStatus
:= xFilterOnOff);
```

```
//Updating the IO for the valve and UV filter
V01_activate := ValveController.xValve;
UV01_activate := ValveController.xUvFilter;

END_PROGRAM
```